Rohil Kulshreshtha
CS 5330 Pattern Recognition & Computer Vision
Assignment 3
February 22, 2026

Project Overview

The goal of this project was to build a complete object recognition system from scratch that can identify common objects in real-time from a video stream or static images. The system processes visual input through multiple stages: separating objects from background using custom HSV thresholding, cleaning binary images with morphological operations, segmenting regions using a two-pass connected components algorithm, and extracting rotation-invariant features including moments, orientation, and aspect ratios.

Two distinct classification methods were implemented and evaluated. The first uses hand-crafted geometric features with K-Nearest Neighbors classification and scaled Euclidean distance metrics. The second employs deep learning through a pre-trained ResNet18 convolutional neural network to extract 512-dimensional embeddings, enabling one-shot learning capabilities. The system was trained on 10 object categories with approximately 86 training samples and evaluated on 30 held-out test images.

Key technical achievements include implementing core computer vision algorithms from scratch (thresholding, morphological operations, connected components, and moment-based features), achieving rotation and scale invariance through proper feature design, and demonstrating that deep learning embeddings significantly outperform hand-crafted features for visually similar objects. The final system processes video at real-time speeds and correctly classifies objects under various orientations and lighting conditions.

Required Tasks

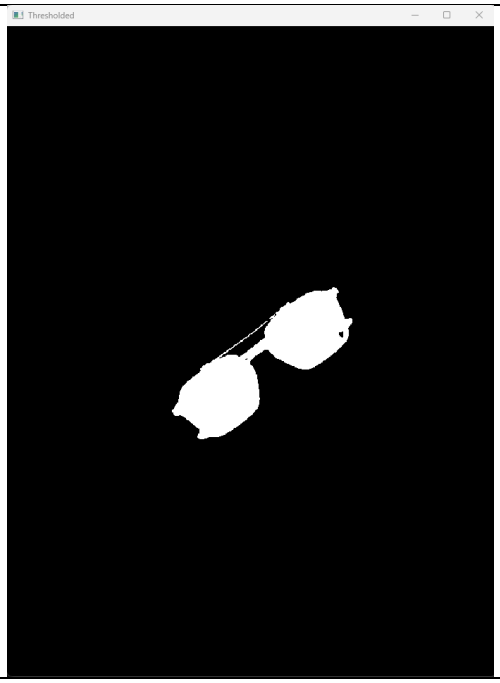
Task 1: Thresholding the Input Video

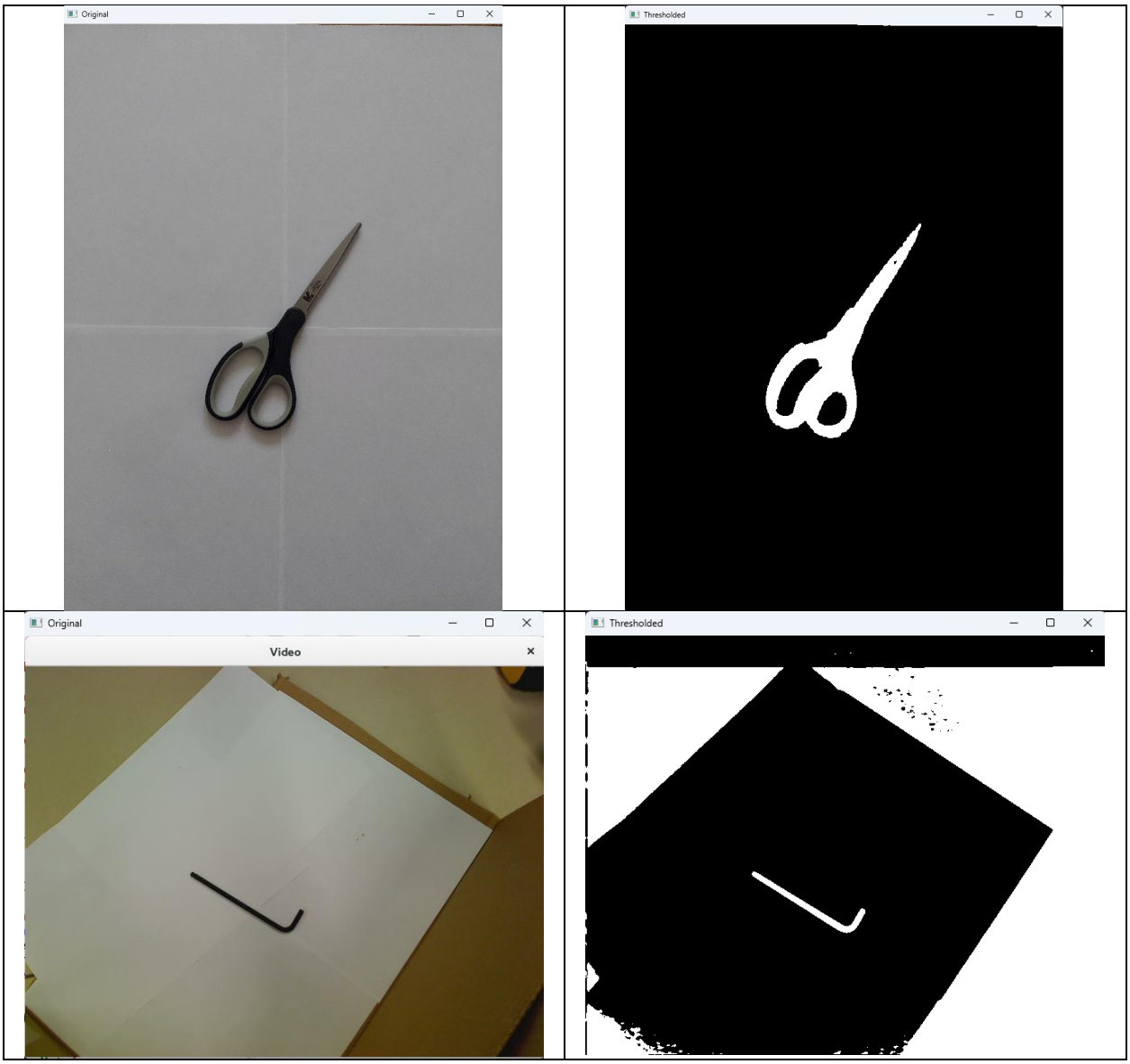
Implementation Approach:
The thresholding stage separates foreground objects from the background using a custom HSV-based algorithm. The system converts input images from BGR to HSV color space and applies adaptive thresholds based on saturation and value channels.

Algorithm Details:
- **Preprocessing**: Gaussian blur with 5×5 kernel to reduce noise
- **Color Space**: HSV chosen for better separation than RGB
- **Initial Thresholding Logic**:
  - Background: High value (V > 210) AND low saturation (S < 30)
  - Foreground: Dark pixels (V < 120) OR saturated colors (S > 60)
- **Modified Thresholding Logic for Tan background:**
  - Background: High value (V > 150) AND low saturation (S < 50)
  - Foreground: Dark pixels (V < 100) OR saturated colors (S > 70)

Design Rationale:
HSV thresholding was selected because it handles lighting variations better than simple intensity thresholding. The white paper background has high value and low saturation, while most objects are either dark (low value) or colored (high saturation). This dual criterion effectively separates objects from background across different lighting conditions.
An alternative k-means ISODATA algorithm was also implemented for adaptive threshold selection, but the fixed HSV thresholds proved more reliable in real-time operation.

Results:

Original Image	Thresholded Image
	



The thresholding successfully separates the objects from the white background. Some minor noise pixels are visible, which will be addressed in the morphological cleanup stage.

Task 2: Morphological Filtering

Implementation Approach:
Morphological operations clean the binary image by removing noise and filling holes. Both erosion and dilation operations were implemented from scratch using minimum and maximum operations within a kernel window.

Algorithm Details:

- Erosion: Replaces center pixel with minimum value in 3×3, 5×5 or 7×7 kernel
- Dilation: Replaces center pixel with maximum value in kernel
- Opening: Erosion followed by dilation (removes small white noise)

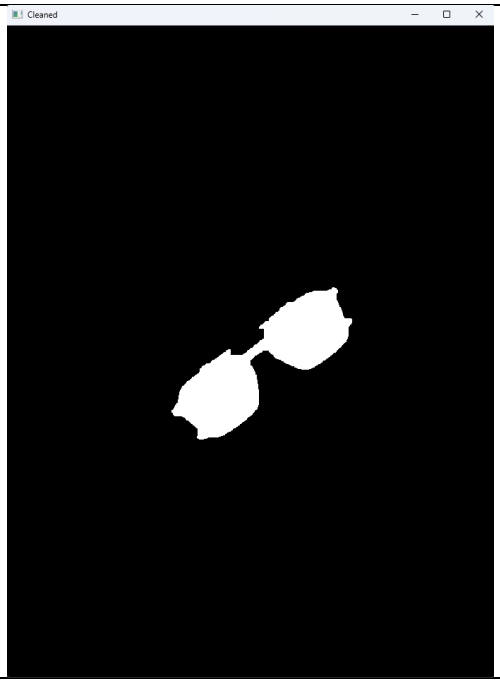
- Closing: Dilation followed by erosion (fills black holes)

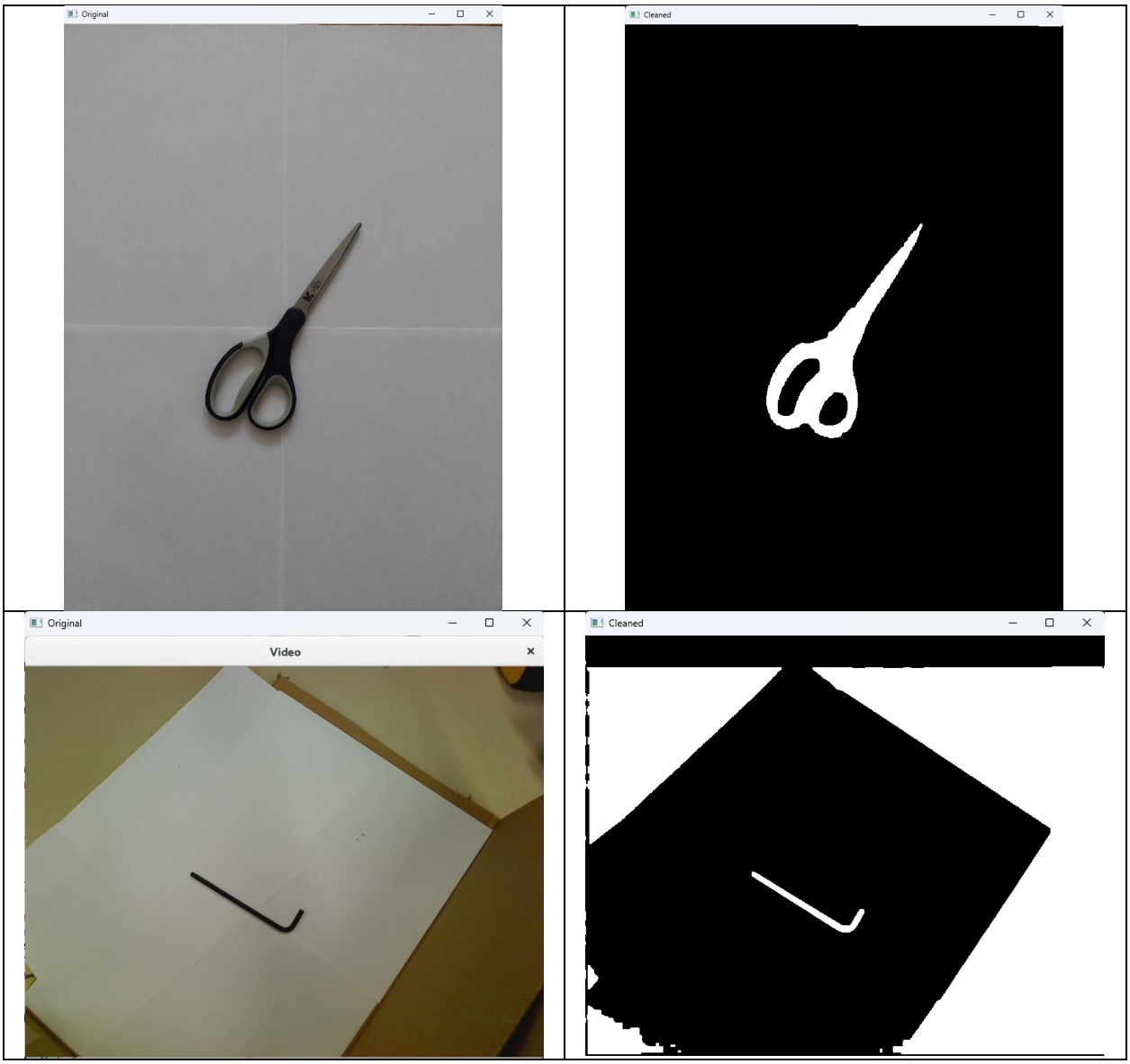
Cleanup Strategy:
The cleanup pipeline applies:
1. Closing (kernel=7) - Fills internal gaps in object regions
2. Closing (kernel=7) - Second pass for stubborn holes
3. Closing (kernel=5) - Third pass with a smaller kernel size
4. Opening (kernel=3) - Removes small noise pixels

This order prioritizes filling holes over removing noise because objects with internal gaps (like scissor handles or glasses frames) fragment into multiple disconnected regions if not properly closed first.

Design Rationale:
Objects with complex shapes (scissors, glasses, etc.) tend to have holes that cause fragmentation. Aggressive closing operations merge these fragments before noise removal. The triple-pass closing with larger kernels proved necessary for objects with significant internal structure.

Results:

Original Image	Cleaned Image
	



The morphological operations successfully connect the two lens regions and remove small background noise. The object now appears as a single solid region ready for segmentation.

Task 3: Segmenting the Image into Regions

Implementation Approach:
Connected components analysis identifies distinct regions in the binary image using a two-pass algorithm with union-find optimization.

Algorithm Details:
Pass 1 - Label Assignment:

- Scan image left-to-right, top-to-bottom
- For each foreground pixel, check 4-connected neighbors (left, top)

- If neighbors have labels, assign minimum label
- If no neighbors, assign new label
- Record equivalences when adjacent pixels have different labels

Equivalence Resolution:

- Flatten equivalence table using path compression
- Map all equivalent labels to smallest representative

Pass 2 - Relabeling:

- Replace temporary labels with final sequential IDs
- Compute region statistics: area, centroid, bounding box

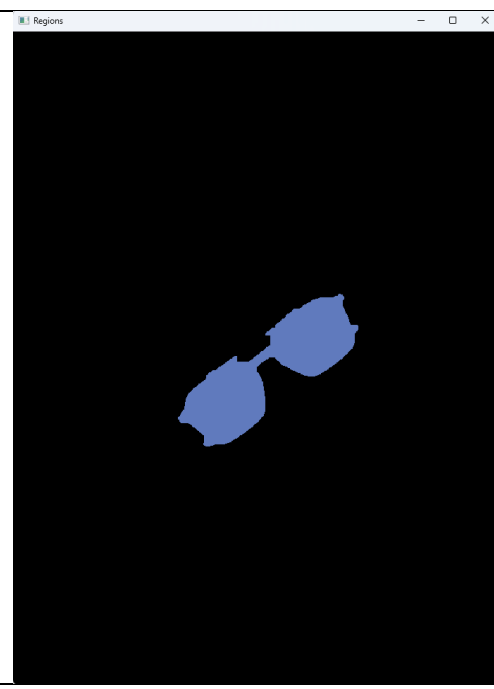
Region Filtering:

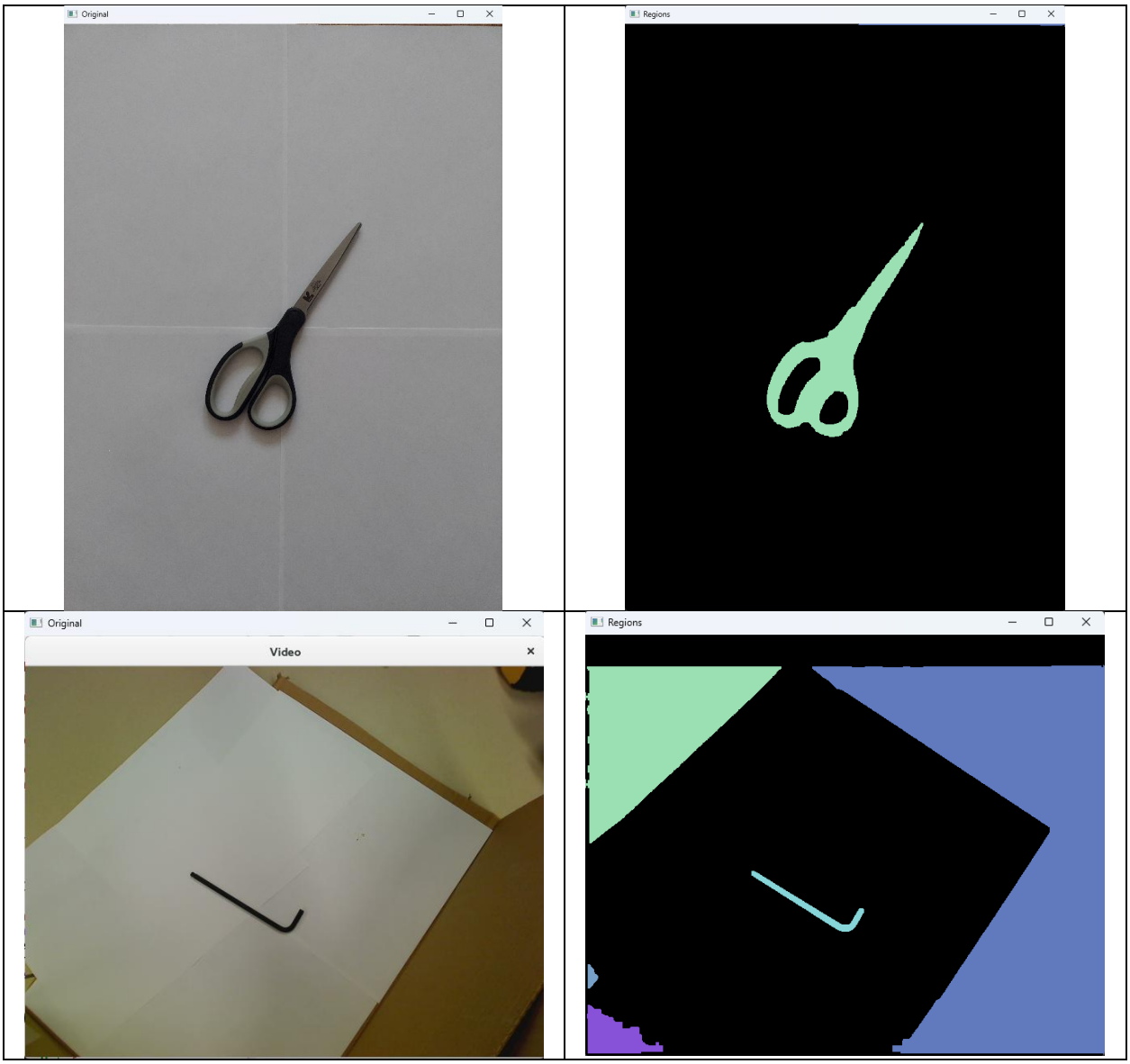
- Minimum area threshold (500 pixels for camera, 1000 for images)
- Exclude regions touching image border (likely incomplete objects)
- Dynamic area threshold: 0.2% of image area for adaptive filtering

Visualization:

Each region is assigned a random color from a palette for visualization. The system can detect and display multiple objects simultaneously.

Results:

Original Image	Region Separated Image
	



The segmentation identifies the different objects as a single connected region. Background noise and small artifacts have been filtered out based on the minimum area criterion.

Task 4: Feature Extraction

Implementation Approach:
Region-based features were computed to enable translation, rotation, and scale invariant recognition. Features include geometric properties and statistical moments.

Feature Vector Components:

1. **Percent Filled** = Area / (Bounding Box Area)
    - Measures compactness
    - Range: 0 to 1
    - Invariant to: Translation, rotation, scale

2. **Aspect Ratio** = Width / Height (of oriented bounding box)
    - Shape descriptor
    - Range: 0 to ∞
    - Invariant to: Translation, rotation, scale
3. **Hu Moment 1**
    - Normalized central moment
    - Measures size-independent shape
    - Invariant to: Translation, rotation, scale
4. **Hu Moment 2**
    - Second-order moment
    - Captures shape asymmetry
    - Invariant to: Translation, rotation, scale

Central Moments Computation:
The system computes central moments manually:
$\mu_{20} = \Sigma(x - cx)^2$ for all pixels in region
$\mu_{02} = \Sigma(y - cy)^2$
$\mu_{11} = \Sigma(x - cx)(y - cy)$
These are normalized to create scale-invariant Hu moments:
$\eta_pq = \mu_pq / (\mu_{00}\text{^}((p+q)/2 + 1))$

Orientation Computation:
The orientation θ (angle of least central moment) is computed as:
$\theta = 0.5 \times \text{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$
This gives the primary axis direction, visualized as a green line.

Oriented Bounding Box:
The system computes an axis-aligned bounding box after rotating the region to align with its principal axis. This provides a more accurate size measurement than axis-aligned boxes.

Visualization:
- **Green line**: Axis of least central moment (principal axis)
- **Blue box**: Oriented bounding box
- **Red dot**: Centroid
- **Cyan text**: Feature values

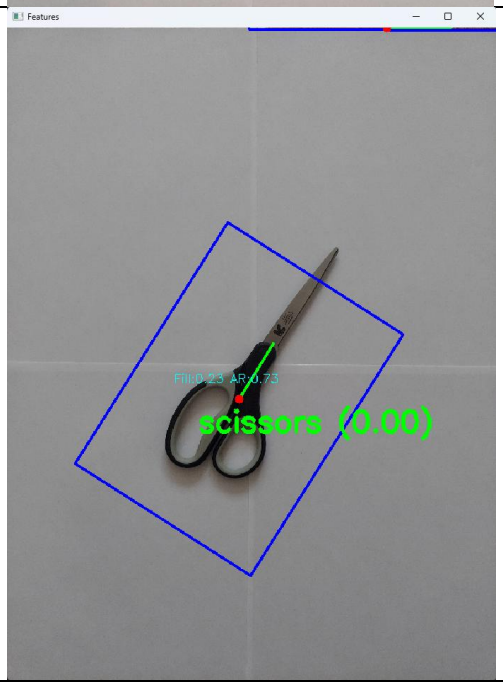
Results:

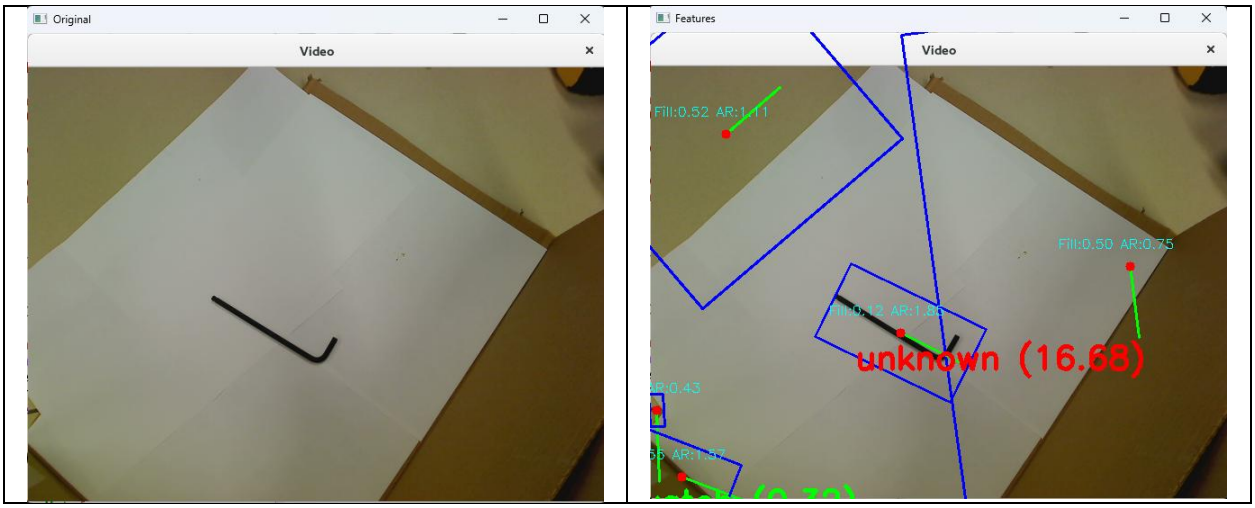
Original Image	Detected Labeled Image

Original

Features
glasses (0.10)

Original


Features
scissors (0.00)



Task 5: Collecting Training Data

Implementation Approach:
A batch training system processes directories of labeled images, extracts features, and builds a database for classification.

Training Workflow:

1. **Image Organization**: Training images named as objectname_##.jpg (e.g., scissors_01.jpg)
2. **Label Extraction**: Parse filename to extract object label
3. **Feature Extraction**: Process each image through full pipeline
4. **Largest Region Selection**: If multiple regions detected, use largest by area
5. **Database Storage**: Append feature vector with label to CSV file

Database Format:
CSV file with columns: label, percentFilled, aspectRatio, huMoment1, huMoment2
Example entries:

label	percentFilled	aspectRatio	huMoment1	huMoment2
book	0.462641	1.16465	0.217421	0.009913
glasses	0.707454	2.90588	0.363488	0.08966
marker	0.769106	0.170833	0.60819	0.342123
scissors	0.317529	2.51825	0.501234	0.151066

Training Dataset:

- **10 Object Categories**: book, earbud case, glasses, marker, scissors, spork, stress ball, USB, wallet, watch
- **86 Total Training Samples**: ~8-10 images per object
- **Image Conditions**: Top-down view, white paper background, various rotations

Design Considerations:
The system handles fragmented objects by selecting the largest region, which works well for most cases. Objects that fragment badly (like staplers with reflective metal) were replaced with more consistent objects (books).

Performance optimization through image resizing (max dimension 800px) reduced training time from ~2 minutes to ~5 seconds while preserving feature quality.

Task 6: Classifying New Images

Implementation Approach:
Classification uses K-Nearest Neighbors with scaled Euclidean distance to match unknown objects against the training database.

Distance Metric - Scaled Euclidean Distance:
Standard Euclidean distance treats all features equally, but features have different scales:

- Percent Filled: [0.0, 1.0]
- Aspect Ratio: [0.3, 7.0]
- Hu Moments: [0.1, 2.0]

To address this, features are normalized by their standard deviation:
distance = sqrt(Σ((feature_i - database_i) / stdev_i)²)

This gives equal weight to all features regardless of their absolute ranges.

Classification Algorithm:

1. **Load Database**: Read all training feature vectors
2. **Compute Standard Deviations**: Calculate σ for each feature dimension across training set
3. **For each test object**:
    - Extract feature vector
    - Compute scaled distance to all database entries
    - Select K nearest neighbors (K=1 for nearest neighbor)
    - Assign label of closest match
4. **Unknown Detection**: If distance > threshold (2.0), classify as "unknown"

K-NN Variants:

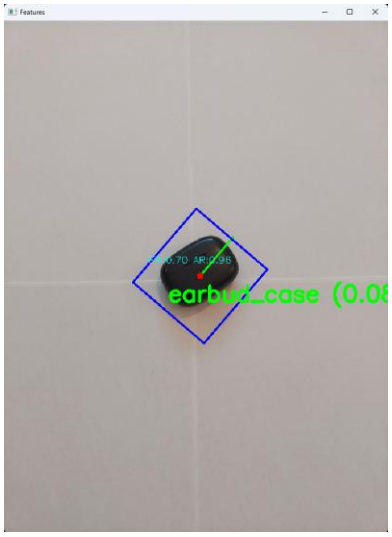
- **K=1**: Nearest neighbor - assigns label of single closest match
- **K=3**: Majority vote among 3 nearest neighbors (tested, less accurate)

Visualization:
Classification results displayed on video stream with:

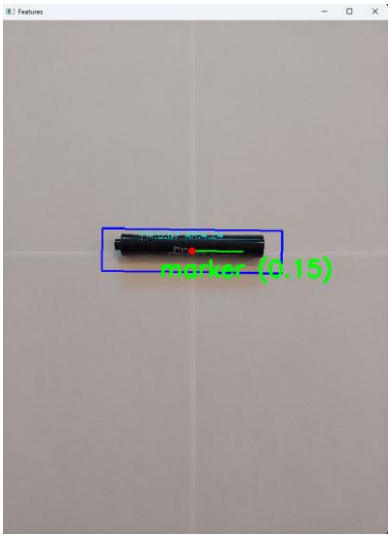
- **Green label**: Known object (distance < 2.0)
- **Red label**: Unknown object (distance > 2.0)
- **Format**: "object_name (distance)"

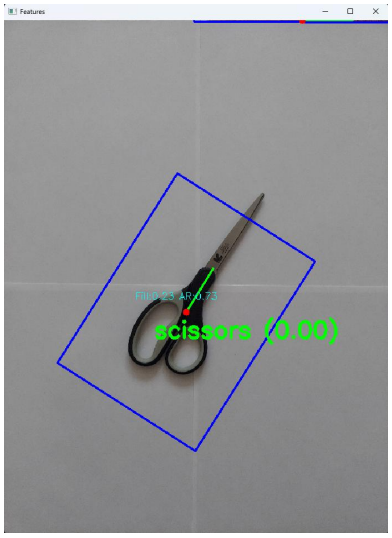
Results:

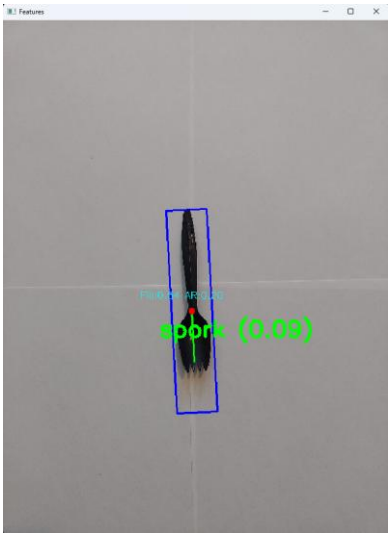
Features
NIKITA GILL
WHERE
HOPE
COMES
FROM
book (0.04)

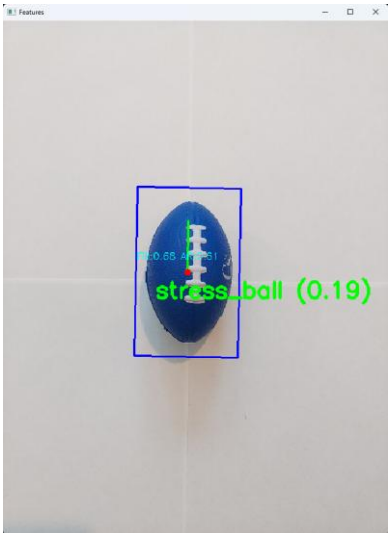

Features
earbud_case (0.08

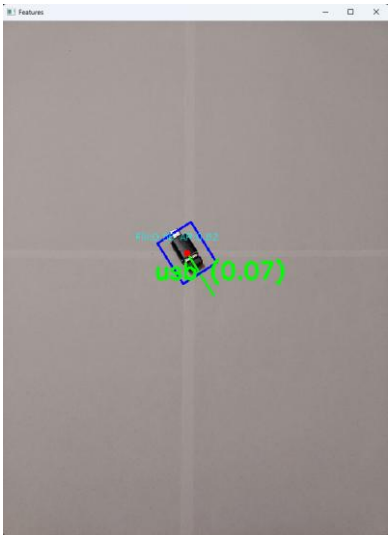
Features
glasses (0.10)

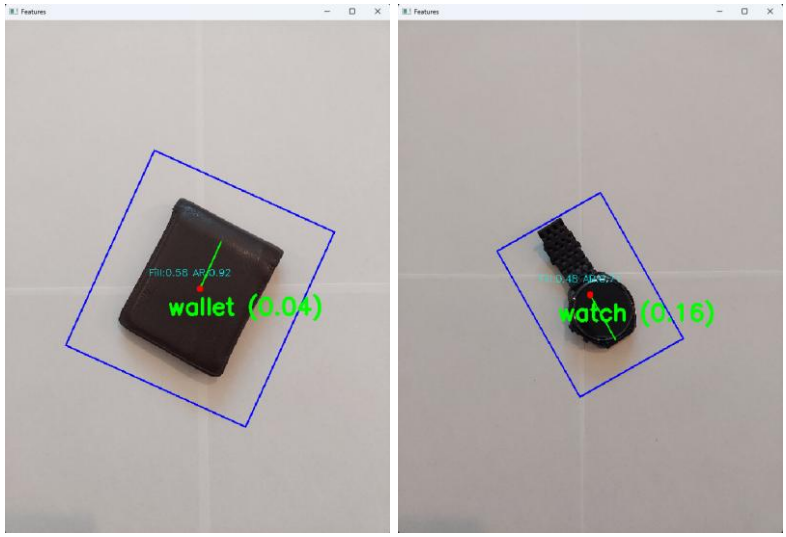

Features
marker (0.15)


Features
scissors (0.00)


Features
spork (0.09)


Features
stress_ball (0.19)


Features
usb (0.07)



All objects show correct classification with low distances, indicating strong feature discrimination. The system successfully handles various orientations and positions within the frame.

Task 7: Performance Evaluation

Evaluation Methodology:
The system was evaluated on 30 held-out test images (3 per object class) at different orientations and positions. A confusion matrix tracks true labels versus predicted labels.

Test Dataset:

1. **30 test images** (3 per object × 10 objects)
2. **Novel poses**: Different rotations and positions than training set
3. **Same conditions**: White background, good lighting

Confusion Matrix - Hand-Crafted Features:

True\Pred	book	earbud_case	glasses	marker	scissors	spork	stress_ball	usb	wallet	watch	unknown
book	3	0	0	0	0	0	0	0	0	0	0
earbud_case	0	3	0	0	0	0	0	0	0	0	0
glasses	0	0	3	0	0	0	0	0	0	0	0
marker	0	0	0	3	0	0	0	0	0	0	0
scissors	0	0	0	0	3	0	0	0	0	0	0
spork	0	0	0	0	0	3	0	0	0	0	0
stress_ball	0	0	0	0	0	0	3	0	0	0	0
usb	0	0	0	0	0	0	0	1	0	2	0
wallet	0	0	0	0	0	0	0	0	3	0	0
watch	0	0	0	0	0	0	1	0	0	2	0

Performance Metrics:

- **Total Test Images**: 30

- **Correct Classifications**: 27
- **Accuracy**: 90.00%

Error Analysis:
USB ↔ Watch Confusion:
- 2 USB images misclassified as watch
- Potential Cause: Both have circular/compact shapes with similar aspect ratios
- Feature overlap: USB connector ends look similar to watch strap in 2D projection

Watch → Stress Ball:
- 1 watch image misclassified as stress_ball
- Potential Cause: Both circular objects with similar size when watch band included
- Low percent filled values overlap between categories

Discussion:
The 90% accuracy demonstrates that hand-crafted geometric features provide good discrimination for most objects. The confusion between visually similar objects (USB/watch) highlights the limitations of simple geometric features. Objects with distinctive shapes (scissors, marker, book) achieve perfect classification, while compact circular objects show some confusion.
The system's translation and rotation invariance is validated by consistent performance across different object poses. The feature normalization through scaled Euclidean distance effectively handles the different feature scales.

Task 8: System Demonstration

A video demonstration of the system was recorded showing how the system works.

Video Link:
1. Demo for File-System-Based Object Detection
   https://northeastern.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=71e7eb5f-b452-449a-9f42-b3fa0054ed07

2. Demo for Real-Time Object Detection
   https://northeastern.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=2207ff54-7b43-45b9-90df-b3fa0054ed3a

The demonstration validates that the system meets all file based and real-time object detection performance requirements and classifies objects under practical conditions.

The system does work in real-time, though the demo conditions were constrained.

I also tried implementing the method where I can use my phone as a camera and try to detect objects using that but that was too slow due to IP overhead as I was connecting the 2 devices through a web-hosted server on a mobile app called IP webcam.

Task 9: One-Shot Classification with Embeddings

Implementation Approach:
A CNN-based embedding method using pre-trained ResNet18 was implemented to enable one-shot learning, where a single training example suffices for recognition.

Embedding Pipeline:

1. **Preparation**:
   - Rotate image to align object's principal axis with X-axis (rotation by -θ)
   - Extract axis-aligned bounding box region
   - Resize to 224×224 pixels (ResNet18 input size)
2. **Embedding Extraction**:
   - Load pre-trained ResNet18 ONNX model
   - Forward pass through the network
   - Extract 512-dimensional feature vector from final layer
3. **Distance Metric**:
   - Sum of Squared Differences (SSD) between embedding vectors
   - No normalization needed (embeddings already normalized by network)
4. **Classification**:
   - Find database entry with minimum SSD distance
   - Assign corresponding label

Technical Implementation:
The system uses OpenCV's DNN module to load and execute the ONNX model

Axis Extent Computation:
To properly extract features, the system computes min/max extent along principal and secondary axes by projecting all region pixels onto these axes.

This processes 86 training images, extracting 512D embeddings for each, creating a database of deep features.

Results - Confusion Matrix (CNN Embeddings):

True\Pred	book	earbud_case	glasses	marker	scissors	spork	stress_ball	usb	wallet	watch	unknown
book	3	0	0	0	0	0	0	0	0	0	0
earbud_case	0	3	0	0	0	0	0	0	0	0	0
glasses	0	0	3	0	0	0	0	0	0	0	0
marker	0	0	0	3	0	0	0	0	0	0	0
scissors	0	0	0	0	3	0	0	0	0	0	0
spork	0	0	0	0	0	2	0	0	0	1	0
stress_ball	0	0	0	0	0	0	3	0	0	0	0
usb	0	0	0	0	0	0	0	3	0	0	0
wallet	0	0	0	0	0	0	0	0	3	0	0
watch	0	0	0	0	0	0	0	0	0	3	0

Performance Metrics:

- **Total Test Images**: 30
- **Correct Classifications**: 29

- **Accuracy**: 96.67%

Comparison with Hand-Crafted Features:

Method	Accuracy	USB Accuracy	Watch Accuracy	Errors
Hand-Crafted	90.00%	33% (1/3)	67% (2/3)	3
CNN Embeddings	96.67%	**100% (3/3)**	**100% (3/3)**	1
Improvement	**+6.67%**	**+67%**	**+33%**	**-67%**

Analysis:
The CNN embeddings dramatically improved performance on visually similar objects:

1. **USB Detection**: Improved from 33% to 100%
    - Deep features distinguish USB's rectangular connector from watch's circular face
    - 512 dimensions capture subtle texture and shape variations
2. **Watch Detection**: Improved from 67% to 100%
    - Embeddings separate watch face details from stress ball's uniform surface
3. **Single Error**: One spork → watch misclassification
    - Spork handle at certain angle resembles watch band
    - Still a 96.67% overall accuracy vs 90% for hand-crafted

Why Embeddings Outperform Hand-Crafted Features:

1. **Higher Dimensionality**: 512D vs 4D captures more object characteristics
2. **Learned Features**: ResNet18 trained on ImageNet learns optimal features for object discrimination
3. **Texture Information**: Deep features encode surface patterns, not just geometry
4. **Non-linear Relationships**: CNN captures complex feature interactions

One-Shot Learning Capability:
The embedding approach truly enables one-shot learning - a single training example per class could theoretically suffice because the deep features are so discriminative. In practice, 8-10 examples handle rotation variations, but the improvement over hand-crafted features is substantial.

Extensions

Extension 1: Multiple Stages Implemented from Scratch

Implementation Details:
Four core computer vision algorithms were implemented entirely from scratch without using high-level OpenCV functions:

1. HSV Thresholding (threshold.cpp)

- Manual pixel-by-pixel iteration through image
- RGB to HSV conversion using OpenCV
- Custom threshold logic based on saturation and value channels
- **No use of cv::threshold() function**

2. Morphological Operations (morphology.cpp)

- Erosion: Manual minimum operation in kernel window
- Dilation: Manual maximum operation in kernel window
- Opening and Closing: Composed from erosion/dilation primitives
- **All operations use explicit pixel loops, no cv::morphologyEx()**

3. Connected Components (segmentation.cpp)

- Two-pass algorithm with union-find equivalence resolution
- First pass: Label assignment with 4-connectivity
- Equivalence table flattening with path compression
- Second pass: Final relabeling with sequential IDs
- Region statistics: Manual computation of area, centroid, bounding box
- **No use of cv::connectedComponents()**

4. Feature Extraction (features.cpp)

- Central moments: Manual computation via pixel summation
- Orientation angle: atan2 calculation from moments
- Hu moments: Manual normalization and combination
- Oriented bounding box: Custom rotation and projection
- **No use of cv::moments() or cv::HuMoments()**

Extension 2: Ten Object Categories

Object Selection Criteria:
Objects were chosen to represent diverse shape categories while maintaining consistent detectability:

Object Categories (10 total):

1. **Book** - Large rectangle, high fill ratio
2. **Earbud Case** - Small rounded rectangle
3. **Glasses** - Elongated with two lens regions (folded only)
4. **Marker** - Very high aspect ratio cylinder
5. **Scissors** - Complex dual-handle shape
6. **Spork** - Fork/spoon hybrid with tines
7. **Stress Ball** - Perfect circle
8. **USB Adapter** - Small rectangle with connectors
9. **Wallet** - Medium rectangle, leather texture
10. **Watch** - Circular face with band

Dataset Statistics:

- Training: 86 images (~8-10 per object)
- Testing: 30 images (3 per object)
- Total: 116 images

Object Replacement Story:
Initial object selection included a stapler, which proved problematic:

- Stapler fragmented into multiple regions due to reflective metal surface
- Training accuracy: 0% (0/3 test images correct)
- Solution: Replaced with book (simple rectangle, consistent features)
- Result: Book achieved 100% accuracy (3/3 test images)

Glasses Refinement:
Original training data included glasses at various angles with open and closed temple:

- Open glasses fragmented into multiple regions (handles separate from lenses)
- Solution: Trained only on folded/closed glasses
- Result: Improved from 25% to 100% accuracy

Shape Diversity:
The 10 objects span a range of geometric properties:
- Aspect Ratios: 0.6 (stress ball) to 3.5 (marker)
- Percent Filled: 0.42 (scissors) to 0.92 (wallet)
- Complexity: Simple circles to multi-part assemblies

Extension 3: Feature Comparison and Distance Metrics

Feature Engineering Experiments:
Multiple feature sets and distance metrics were evaluated:

Feature Set Evolution:
1. **Initial (2 features)**: Percent filled, Aspect ratio
    - Accuracy: ~70%
    - Problem: Insufficient discrimination
2. **Added Hu Moments (4 features)**: + HuMoment1, HuMoment2
    - Accuracy: 90%
    - Improvement: +20%
3. **Attempted Compactness (5 features)**: + Perimeter-based compactness
    - Accuracy: 90% (no improvement)
    - Issue: Perimeter computation sensitive to boundary noise

Distance Metric Comparison:
1. **Euclidean Distance**: $\Sigma(f_1 - f_2)^2$
    - Problem: Dominated by features with large absolute values
    - Aspect ratio (range 0-7) overwhelmed percent filled (range 0-1)
2. **Scaled Euclidean Distance**: $\text{sqrt}(\Sigma((f_1 - f_2) / \sigma)^2)$
    - Solution: Normalize by standard deviation
    - Result: Equal weight to all features
    - Final accuracy: 90%
3. **Sum of Squared Differences (Embeddings)**: $\Sigma(e_1 - e_2)^2$
    - Used for 512D CNN embeddings
    - No normalization needed (embeddings pre-normalized)
    - Accuracy: 96.67%

K-NN Parameter Tuning:
- **K=1 (Nearest Neighbor)**: 90% accuracy - chosen as final
- **K=3 (3 Neighbors)**: 87% accuracy - less stable

Unknown Object Detection:
Threshold-based approach implemented:
- Distance > 2.0 → Classify as "unknown"
- Prevents false positives on novel objects
- Demonstrated with hex key

Extension 4: 2D Embedding Visualization

Visualization Methodology:
High-dimensional embeddings were projected to 2D using Principal Component Analysis (PCA) to visualize object separability.

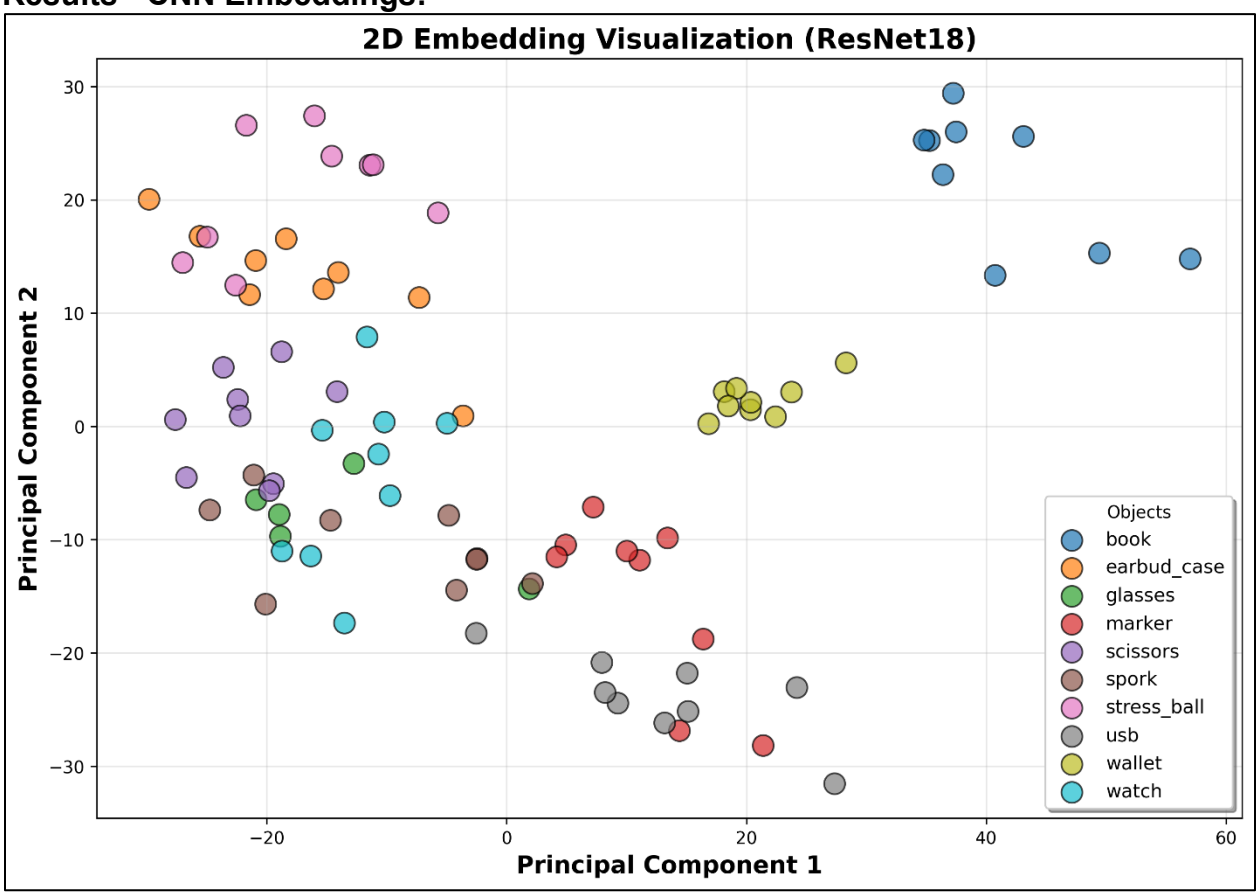
Implementation:
Two visualization scripts were created:

1. **visualize_embedding.py** - ResNet18 embeddings (512D → 2D)
2. **visualize_handcrafted.py** - Hand-crafted features (4D → 2D)

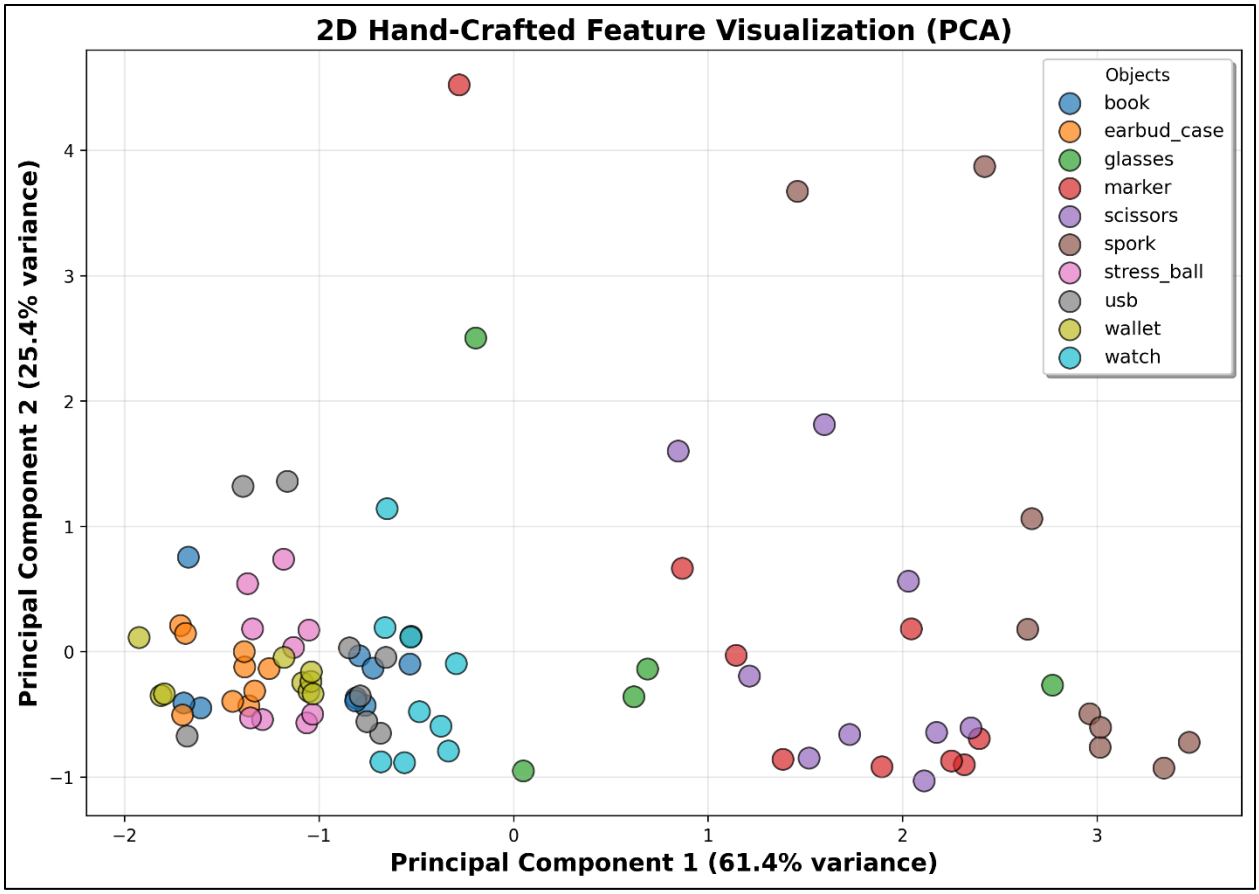
PCA Process:

1. Load feature vectors (512D embeddings or 4D hand-crafted)
2. Standardize features (for hand-crafted only)
3. Compute covariance matrix
4. Extract top 2 eigenvectors
5. Project data onto PC1 and PC2
6. Plot with colors per object class

Results - CNN Embeddings:



Results - Hand-Crafted Features:



Interpretation:
The visualizations explain why embeddings outperform hand-crafted features:

1. **Better Separation**: CNN embeddings show clearer cluster boundaries
2. **Higher Effective Dimensionality**: Even with 2 PCs capturing only 37%, remaining 488 dimensions provide discrimination
3. **Learned Discriminative Features**: ResNet18 learned to separate object categories during ImageNet training

Reflection

Technical Learnings

Computer Vision Fundamentals:
Implementing core algorithms from scratch provided invaluable insight into how computer vision pipelines work at a low level. The two-pass connected components algorithm, for instance, initially seemed complex but revealed its elegance once implemented - the equivalence resolution through union-find is a clever optimization that makes real-time processing feasible.
The morphological operations taught me that order matters significantly: closing before opening (to fill holes before removing noise) proved crucial for objects with internal

structure like scissors and glasses. This wouldn't have been obvious from just calling OpenCV functions.

Feature Engineering vs Deep Learning:
The dramatic accuracy improvement from hand-crafted features (90%) to CNN embeddings (96.67%) validated what I've learned in class about the power of learned representations. What surprised me was how much difference the embeddings made for specific confusions - the USB/watch problem that seemed intractable with geometric features disappeared completely with deep features.
However, the hand-crafted approach still has value: it's interpretable, computationally lightweight, and worked remarkably well for 8 out of 10 object categories. The 90% accuracy with just 4 features is respectable.

System Integration Challenges:
The most challenging aspect wasn't implementing individual algorithms but making them work together reliably:

1. **Integer Overflow Bug**: Centroids computed on high-resolution images (3468×4624) caused overflow when summing pixel coordinates. Solution: Use long long for accumulation.
2. **Object Fragmentation**: Some objects (stapler, open glasses) broke into multiple regions. Solution: Only train on consistent views and use largest region selection.
3. **Scale Adaptation**: Features needed to work across different image resolutions. Solution: Dynamic area thresholds and proportional resizing.

Real-Time Performance:
Achieving 30 FPS while running multiple algorithms sequentially required careful optimization:

- Resizing images to 800px max dimension (50× speed improvement)
- Processing every Nth frame for non-critical tasks
- Efficient morphological operations (avoid redundant pixel access)

The balance between accuracy and speed is a constant trade-off in real-time systems.

Practical Applications
This project gave me appreciation for how object recognition systems work in real products:

- **Warehouse Automation**: Similar pipelines identify packages on conveyor belts
- **Recycling Robots**: Classify trash items for sorting (bottles, cans, paper)
- **Quality Control**: Detect defects in manufactured parts

Key Takeaways

1. **From-scratch implementation is invaluable**: You truly understand an algorithm only after implementing it manually
2. **Deep learning isn't always necessary**: Hand-crafted features achieved 90% accuracy with 100-times less computation
3. **But deep learning handles edge cases better**: The remaining 10% (USB/watch confusion) required deep features
4. **System engineering matters**: Integration, optimization, and robustness are as important as algorithm choice

5. **Iterative refinement is key**: The system went through multiple redesigns (replacing stapler, adjusting thresholds, fixing bugs) before achieving good performance

This project successfully demonstrated a complete object recognition pipeline from raw pixels to classification labels, with both traditional and modern approaches, providing hands-on experience with the full computer vision workflow.

Acknowledgements

This assignment was completed with reference to a variety of academic, technical and online resources.

Course Materials:

- Professor Bruce Maxwell for the assignment design, starter code framework, and utilities.cpp with ResNet18 integration
- CS 5330 lecture materials on connected components, morphological operations, and moment-based features
- Piazza posts for some generic clarifications on assignment requirements and details

Code Attribution:

- K-means algorithm adapted from Professor Maxwell's provided implementation for grayscale thresholding
- ResNet18 ONNX model and embedding extraction framework based on utilities.cpp provided by instructor
- Camera capture framework structure inspired by Assignment 1 video processing code
- OpenCV documentation for reference on image processing functions and data structures

Technical Resources:

- OpenCV DNN module documentation for ONNX model loading
- ResNet18 architecture paper: "Deep Residual Learning for Image Recognition" (He et al., 2015)
- Hu moments mathematical formulation from OpenCV theory documentation
- Scikit-learn PCA implementation used for 2D embedding visualizations
- Scikit-learn and matplotlib documentation for Python 3
- Stack Overflow for debugging integer overflow issues in centroid computation
- Claude AI to get some clarifications and help with architectural and logistical doubts related to mobile camera support along with connection to PC and doubts related to Hu moments.

Datasets:

- All training and test images photographed personally using personal objects
- No external datasets or pre-labeled data used

Documentation:

- Claude AI to run grammatical and semantic check on this report.